

09 — Fine-Tuning Approaches

Time: ~6-7 hours | **Level:** Advanced

What you'll learn:

- Why fine-tune? Transfer learning explained
- Full fine-tuning vs feature extraction
- Parameter-Efficient Fine-Tuning (PEFT): the modern approach
- LoRA: Low-Rank Adaptation (the most popular PEFT method)
- QLoRA: LoRA + 4-bit quantisation (what we use in our project)
- Other PEFT methods: Prefix Tuning, Adapters, IA3
- RLHF/DPO: aligning models with human preferences
- Choosing the right approach for your use case

Prerequisites: Notebooks 07-08 (Transformers, HuggingFace)

The Big Picture

Pre-trained model (knows language) + Your data (specific domain)
↓ ↓
Fine-tuning = Domain-specific model

```
In [ ]: import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt

torch.manual_seed(42)
```

1. Transfer Learning — Why Fine-Tune?

Training a model from scratch requires:

- **Data:** trillions of tokens (internet-scale)
- **Compute:** thousands of GPUs for weeks
- **Cost:** millions of dollars

Fine-tuning leverages a pre-trained model's knowledge:

Approach	What's Trained	Data Needed	Compute	When to Use
Train from scratch	Everything	Trillions of tokens		Never (unless you're OpenAI)
Full fine-tuning	All parameters	10K-1M examples	\$\$\$	When you have lots of data + GPU
PEFT (LoRA)	< 1% of params	1K-100K examples	\$	Best default for most cases
Prompt engineering	Nothing	0 (examples in prompt)	¢	Quick experiments

Why not just use prompt engineering?

It works for simple cases, but:

- Eats context window (few-shot examples are long)
- Can't encode deep domain knowledge
- Inconsistent outputs across similar queries
- No ability to learn patterns from large datasets

2. Full Fine-Tuning

Update ALL parameters of the model on your dataset.

The problem:

- Phi-3.5-mini has **3.8 billion** parameters
- In float32: $3.8\text{B} \times 4 \text{ bytes} = 15.2 \text{ GB}$ just for the model
- Training also needs: gradients (15.2 GB) + optimizer states (30.4 GB for Adam)
- **Total: ~61 GB VRAM** — that's 2 × A100 40GB GPUs

Another problem — catastrophic forgetting:

If you update all parameters on a small dataset, the model "forgets" its pre-trained knowledge.

```
In [ ]: # — Visualise the parameter cost —————

models = {
    'GPT-2': 0.124,
    'BERT-Large': 0.340,
    'Phi-3.5-mini': 3.8,
    'LLaMA-3-8B': 8.0,
    'LLaMA-3-70B': 70.0,
    'GPT-3': 175.0
```

```

}

fig, ax = plt.subplots(figsize=(10, 5))
bars = ax.barh(list(models.keys()), list(models.values()), color='steelblue')
ax.set_xlabel('Billion Parameters')
ax.set_title('Model Sizes – Full Fine-Tuning Requires VRAM for All Parameter')
ax.set_xscale('log')

for bar, (name, size) in zip(bars, models.items()):
    vram_gb = size * 4 * 4 # params × 4 bytes × 4 (model+grad+adam)
    ax.text(bar.get_width() * 1.1, bar.get_y() + bar.get_height()/2,
            f'{size}B → ~{vram_gb:.0f}GB VRAM', va='center', fontsize=9)

plt.tight_layout()
plt.show()

print('💡 Full fine-tuning Phi-3.5 needs ~61 GB VRAM.')
print('    A consumer GPU (RTX 4090) has only 24 GB.')
print('    This is why PEFT methods like LoRA exist.')

```

3. LoRA — Low-Rank Adaptation

The key insight: When fine-tuning, weight updates (ΔW) tend to be **low-rank** — they don't need the full dimensionality of the weight matrix.

Instead of updating the full weight matrix $W \in \mathbb{R}^{d \times d}$:

$$W_{\text{new}} = W_{\text{original}} + \Delta W$$

LoRA decomposes the update into two small matrices:

$$\Delta W = B \cdot A \quad \text{where } B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d}$$

Where r (rank) is tiny: typically 8, 16, or 32.

Parameter savings:

- Original ΔW : $d \times d = d^2$ parameters
- LoRA: $d \times r + r \times d = 2dr$ parameters
- For $d = 3072, r = 16$: $3072^2 = 9.4M$ vs $2 \times 3072 \times 16 = 98K$
- **96x fewer parameters!**

```

In [ ]: # — LoRA from Scratch —————

class LoRALayer(nn.Module):
    """
    LoRA: Low-Rank Adaptation of Large Language Models.

    Instead of updating W directly, we learn a low-rank update:
    output = x @ W + x @ B @ A * (alpha / rank)

```

```

W is FROZEN (no gradients). Only A and B are trained.
"""
def __init__(self, original_layer, rank=16, alpha=32):
    super().__init__()
    self.original = original_layer
    self.original.weight.requires_grad = False # FREEZE original weight
    if self.original.bias is not None:
        self.original.bias.requires_grad = False

    d_in = original_layer.in_features
    d_out = original_layer.out_features

    # LoRA matrices (the ONLY trainable parameters)
    self.lora_A = nn.Parameter(torch.randn(d_in, rank) * 0.01) # (d_in, rank)
    self.lora_B = nn.Parameter(torch.zeros(rank, d_out)) # (rank, d_out)
    # B initialised to zero → initial LoRA output is zero → model starts with original output

    self.scaling = alpha / rank # scaling factor

def forward(self, x):
    # Original output (frozen)
    original_output = self.original(x)

    # LoRA output (trainable)
    lora_output = (x @ self.lora_A @ self.lora_B) * self.scaling

    return original_output + lora_output

# Demo: apply LoRA to a linear layer
original = nn.Linear(3072, 3072) # like a query projection in Phi-3
lora_layer = LoRALayer(original, rank=16, alpha=32)

original_params = sum(p.numel() for p in original.parameters())
lora_params = sum(p.numel() for p in lora_layer.parameters() if p.requires_grad)
total_params = sum(p.numel() for p in lora_layer.parameters())

print(f'Original layer parameters: {original_params:>12,}')
print(f'LoRA trainable parameters: {lora_params:>12,}')
print(f'Total parameters (frozen + LoRA): {total_params:>12,}')
print(f'\nTrainable: {lora_params/total_params:.2%} of total')
print(f'Savings: {original_params/lora_params:.0f}x fewer trainable parameters')

# Verify it works
x = torch.randn(2, 3072)
y = lora_layer(x)
print(f'\nInput: {x.shape} → Output: {y.shape} ✅')
print(f'\n💡 B is initialised to ZERO, so lora_output starts at zero.')
print('    The model behaves exactly like the original at the start of training')
print('    Gradients only flow through A and B (98K params instead of 9.4M).')

```

```

In [ ]: # — Visualise LoRA —————

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Full weight matrix
d = 32 # downsized for visualisation

```

```

W = np.random.randn(d, d) * 0.1
axes[0].imshow(W, cmap='RdBu', vmin=-0.3, vmax=0.3)
axes[0].set_title(f'Full Weight Update  $\Delta W$  {d}x{d} = {d*d} params')

# LoRA matrices
r = 4
A = np.random.randn(d, r) * 0.1
B = np.random.randn(r, d) * 0.1

axes[1].imshow(np.hstack([A, np.zeros((d, 2)), B.T]), cmap='RdBu', vmin=-0.3, vmax=0.3)
axes[1].set_title(f'LoRA: A ({d}x{r}) + B ({r}x{d}) \n= {2*d*r} params')
axes[1].axvline(r + 0.5, color='black', linewidth=2)

# Reconstructed
reconstructed = B.T @ A.T # same as A @ B when used correctly
axes[2].imshow(A @ B, cmap='RdBu', vmin=-0.3, vmax=0.3)
axes[2].set_title(f'LoRA Reconstruction: AxB \nRank-{r} approximation of {d}x{d}')

for ax in axes:
    ax.set_xticks([])
    ax.set_yticks([])

plt.suptitle(f'LoRA: {d*d} params → {2*d*r} params ({2*d*r/(d*d):.0%} of original)')
plt.tight_layout()
plt.show()

```

4. QLoRA — Quantisation + LoRA

QLoRA (2023) makes LoRA even more memory-efficient by:

1. **Quantise** base model to 4-bit (NF4 format) → 4x less memory
2. Apply LoRA on top of quantised model
3. Compute gradients in float16/bfloat16 (for stability)

Memory comparison for Phi-3.5-mini (3.8B params):

Method	Model Memory	Total Training Memory
Full fine-tuning (fp32)	15.2 GB	~61 GB
Full fine-tuning (fp16)	7.6 GB	~30 GB
LoRA (fp16 base)	7.6 GB (frozen) + 50 MB (LoRA)	~10 GB
QLoRA (4-bit base)	~2 GB (frozen) + 50 MB (LoRA)	~5 GB

QLoRA lets you fine-tune a 4B parameter model on a single consumer GPU.

This is what our project uses in `configs/training_config.yaml`.

```

In [ ]: # — QLoRA configuration (what our project uses) —————

# This is the EXACT config from our configs/training_config.yaml

```

```

qlora_config = {
    # Quantisation
    'load_in_4bit': True,
    'bnb_4bit_quant_type': 'nf4',          # Normal Float 4-bit (best for norm
    'bnb_4bit_compute_dtype': 'bfloat16', # compute in bf16 for stability
    'bnb_4bit_use_double_quant': True,     # quantise the quantisation consta

    # LoRA
    'r': 16,                             # rank (low = fewer params, high = more expressive
    'lora_alpha': 32,                     # scaling factor (usually 2 * r)
    'lora_dropout': 0.05,
    'target_modules': [                  # which layers to apply LoRA to
        'q_proj',                         # query projection
        'k_proj',                         # key projection
        'v_proj',                         # value projection
        'o_proj',                         # output projection
        'gate_proj',                     # FFN gate (Phi-3 uses gated FFN)
        'up_proj',                       # FFN up projection
        'down_proj',                     # FFN down projection
    ],
}

print('QLoRA Configuration for our Mental Health project:')
print('=' * 55)
for key, value in qlora_config.items():
    print(f' {key}: {value}')

# Calculate parameter count
d_model = 3072 # Phi-3.5-mini hidden size
n_layers = 32  # Phi-3.5-mini layers
rank = 16

# Each target module gets LoRA: d_model * rank * 2 (A and B)
params_per_module = 2 * d_model * rank
lora_modules_per_layer = len(qlora_config['target_modules'])
total_lora_params = params_per_module * lora_modules_per_layer * n_layers

print(f'\n--- Parameter Analysis ---')
print(f'Base model: 3,800,000,000 params (frozen, 4-bit ≈ 2 GB)')
print(f'LoRA per module: {params_per_module:,} params')
print(f'LoRA modules per layer: {lora_modules_per_layer}')
print(f'Total LoRA params: {total_lora_params:,} ({total_lora_params/3.8e9:.1f}x base)')
print(f'LoRA size: ~{total_lora_params * 2 / 1e6:.1f} MB (in fp16)')

```

5. Other PEFT Methods

Method	How It Works	Pros	Cons
LoRA	Low-rank weight updates	Simple, effective, composable	Rank selection
QLoRA	LoRA + 4-bit quantisation	Minimal VRAM	Slightly slower
Prefix Tuning	Learnable tokens prepended to every layer	No weight modification	Limited capacity

Method	How It Works	Pros	Cons
Adapters	Small modules inserted between layers	Modular	Inference overhead
IA3	Learned scaling vectors	Fewer params than LoRA	Less expressive
Full FT	Update all weights	Most expressive	Most expensive

When to use what:

- **Start with QLoRA** — best cost/performance ratio for most cases
- Use **LoRA** if you have a 16-bit GPU (A100, H100)
- Use **Full FT** only with lots of data AND compute
- Use **Prefix Tuning** for very small models or few parameters

6. RLHF & DPO — Alignment

After supervised fine-tuning (what our project does), there's an additional step for chat models:

RLHF (Reinforcement Learning from Human Feedback):

1. Humans rank model outputs (A is better than B)
2. Train a reward model on these rankings
3. Use PPO (reinforcement learning) to optimise the LLM against the reward model

DPO (Direct Preference Optimisation) — simpler alternative:

- Skip the reward model entirely
- Directly optimise from preference pairs
- Less compute, often similar quality

Pre-training → SFT (Supervised Fine-Tuning) → RLHF/DPO
 (language) (task-specific) (human preferences)

For our project:

We do **SFT with QLoRA** — that's the middle step. RLHF/DPO would be a great extension if we had human preference data ("which assessment is better?").

```
In [ ]: # — HuggingFace PEFT Library (what our project uses) —————

# This is the actual code pattern from our src/model/__init__.py
print('=== How our project applies QLoRA ===')
```

```

print()
code = '''
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import get_peft_model, LoraConfig, TaskType

# Step 1: Load model in 4-bit
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)
model = AutoModelForCausalLM.from_pretrained(
    "microsoft/Phi-3.5-mini-instruct",
    quantization_config=bnb_config,
    device_map="auto",
)

# Step 2: Apply LoRA adapters
lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# Output: "trainable params: 22M || all params: 3,800M || trainable%: 0.58%"

# Step 3: Train with HuggingFace Trainer
from transformers import Trainer, TrainingArguments
trainer = Trainer(
    model=model,
    args=TrainingArguments(
        output_dir="outputs/",
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4, # effective batch = 8
        num_train_epochs=3,
        learning_rate=2e-4,
        fp16=True,
        eval_strategy="steps",
        eval_steps=200,
    ),
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
)
trainer.train()

# Step 4: Merge LoRA weights back into base model for serving
merged = model.merge_and_unload()
merged.save_pretrained("outputs/merged_model")
'''
print(code)

```



```
print('💡 This is the complete fine-tuning pipeline in ~30 lines.')
```

```
print(' All the complexity from Notebooks 03-07 is handled by the libraries')
```

7. Choosing Your Approach — Decision Framework

Do you have a GPU with ≥ 24 GB VRAM?

└ No → Use QLoRA (4-bit) or cloud GPU

└ Yes → How much data do you have?

└ < 1000 examples → Few-shot prompting or LoRA rank 4–8

└ 1K–100K examples → LoRA rank 16–64 (our project: 10K examples, rank 16)

└ > 100K examples → Full fine-tuning (if you have A100/H100)

Our project's choices:

- **QLoRA** because: consumer GPU friendly, 10K dataset is small
 - **Rank 16** because: enough capacity for our task, not too many params
 - **Alpha 32** because: 2× rank is the standard starting point
 - **All attention + FFN layers** because: more expressive than attention-only
-

✅ Key Takeaways

1. **Fine-tuning** adapts pre-trained models to your domain
2. **Full fine-tuning** updates all params — expensive, risk of forgetting
3. **LoRA** decomposes weight updates into low-rank matrices (< 1% params)
4. **QLoRA** adds 4-bit quantisation — fits on consumer GPUs
5. **Our project**: QLoRA, rank=16, alpha=32, targeting all attention + FFN layers
6. **RLHF/DPO** is the alignment step AFTER fine-tuning

Next: [10 — Production Fine-Tuning](#) — tying everything together with our actual project